SOVEREIGN · GOVERNED · SELF-HOSTABLE

Sovereign Agentic OS

The product guide — install it, run it, and put every governed workflow to work.

Orchestrated by Data Masterclass • datamasterclass.com • www.sovereign-agentic.com
Chart 0.2.12 (app 0.2.0-alpha.12 • os-ui 0.1.44) • generated 2026-07-07 from commit a2ad0fe

Contents

5 Reference

1 Welcome

> **The Sovereign Agentic OS is orchestrated by Data Masterclass.** Project home and downloads: **www.sovereign-agentic.com**.

This is the guide for the people who use the platform — administrators standing it up, builders governing it, and the teams who create and consume data, knowledge, agents and software inside it. It is written to be read top to bottom once, then kept beside you as a reference.

It is in three parts:

1. **Understand it** — what the Sovereign Agentic OS is, who it is for, and the operating model every part of the product shares.
2. **Get started** — install the platform, open it, and bootstrap your first administrator, real users, and roles.
3. **Use it, tab by tab** — one short chapter per tab, in **sidebar order**, each built around the **golden path**: the exact steps a real person follows in the screen, who plays which role, and how that tab hands off to the others.

A closing **Reference** covers cloud deployment, the security model, component details, and the demo logins.

1.1 What the Sovereign Agentic OS is

The **Sovereign Agentic OS** is a self-hostable, EU-residency platform that assembles roughly two dozen best-in-class, permissively-licensed open-source tools into a single **governed** stack — one place where every business **domain** can create, store, use, document and share data, knowledge, dashboards, agents and software.

It ships as **one umbrella Helm chart** (`charts/sovereign-agentic-os`) that brings the whole platform up on any Kubernetes cluster. The default install is **fully self-contained**: every backend runs inside the chart and a small local model stands in for a cloud LLM, so nothing external — and no API key — is required to see the system work end to end. The same chart scales from a laptop to a sovereign production deployment on **STACKIT** (the EU/Germany cloud); the only difference is a values choice — `mode: bundled | external` — per backend.

1.2 Who it is for

- **Regulated organizations, the public sector, and EU enterprises** that need data residency, full audit trails, and zero dependency on US-controlled cloud or hosted LLM platforms.
- **Teams that cannot send data to hosted AI services** but still want production-grade agentic workflows — RAG agents, a lakehouse, BI, and software delivery — all self-hosted.
- **Data Masterclass participants** building real agentic systems on the *same* production components used in the field, not teaching forks.

1.3 Three principles

1. **Permissive open source only.** Every bundled component is Apache-2.0 / MIT / BSD / PostgreSQL licensed — full code auditability, no proprietary lock-in, the right to host and modify indefinitely. This drove deliberate choices: **Valkey** instead of Redis (relicensed), **OpenSearch** for vector *and* lexical retrieval (no pgvector), **Forgejo** instead of Gitea, **Langfuse** instead of LangSmith.
2. **Security and governance are the operating model, not an add-on.** The platform is *secure by default*: agents have no raw internet, every tool call is authorized, every model call is metered and traced, and no real secret ever lives in git.
3. **Self-hosted, in your region.** The production target is STACKIT Kubernetes with STACKIT Object Storage in **EU01 / Deutschland Süd**; model calls can route to **STACKIT AI Model Serving** so prompts and completions never leave the sovereign boundary.

1.4 How the platform is layered

You can reason about — and enable — the platform in layers:

- **Layer 1 — Agent core.** The runtime: **LangGraph** agents calling **LiteLLM** (the one model + tool gateway, with per-key access control and cost caps), every action traced in **Langfuse**, retrieving over **OpenSearch** (hybrid vector + lexical).
- **Layer 2 — Foundations.** Turning raw data and knowledge into governed products: **OPA** (policy-as-code at the tool boundary), **Docling** (document parsing), **Haystack** (RAG), **Dagster** (orchestration), **dbt** (transforms), **Cube** (the metrics layer), **OpenMetadata** (catalog + lineage).
- **Layer 3 — Self-service.** Query, visualize, and ship: the **Iceberg** lakehouse with **central Trino** as the one governed query engine, **Superset** for dashboards, and **in-cluster Forgejo + Argo CD** for software delivery (git → CI → GitOps).
- **Layer 4 — Science / ML.** Classic ML — **JupyterHub**, **MLflow**, **Featureform**, **KServe** — *opt-in and off by default* (heavier, GPU-oriented).
- **Security baseline** spans every layer: default-deny egress through a single proxy chokepoint, a governed `web_fetch`, OPA tool authorization, externalized secrets, hardened pods.

On top sits the **front door** you open in a browser: the **OS UI** (for everyone), with the stack's operational console embedded inside it under **Platform → Components** — there is no separate admin service to run.

> **Where this release stands.** Layers 1–3 are in place; **Science (Layer 4) is opt-in / off by default**. The **OS UI is v1.0** — every sidebar tab is a real surface, brand-themed, with a light/dark theme (light is the default). The governance spine — OPA, approvals, RLS, promote ladders, roles, audit, MCP (live end-to-end at `https://agentic.datamasterclass.com/api/mcp`), auth, Knowledge, and the Data ingest pipeline — is **fully live**. A few execution and integration surfaces are still being wired: real external tool execution beyond Google Drive / OneDrive, and the Science / Layer-4 ML tab (deferred, opt-in). The Software tab's **in-cluster live-app runner** provisions a real Deployment + Service + Ingress with a per-app live URL. The core is **Apache-2.0**; bundled components keep their own licenses.

2 The operating model

Before the tabs, one idea ties them together. Learn it once and every screen reads the same way.

2.1 Everything is a governed artifact

Whatever you make — a data product, a knowledge workflow, a file, an agent, an app, an ML model, a metric, a connection, a dashboard — it is an **artifact** with the same four attributes: **owner** • **domain** • **type** • **visibility**. And whatever the type, it travels the same lifecycle:

> **Create → Document → Use → Promote** — authored in the OS UI, which scaffolds the real tool underneath (a dbt model, a Cube metric, a Forgejo repo, a KServe service), preview-first, cataloged and audited.

2.2 The sharing ladder

Visibility widens one rung at a time, and each rung is strictly **two-step** — the person who *triggers* the move is never the person who *approves* it:

VISIBILITY	MEANING	WHO TRIGGERS	WHO APPROVES
Personal	the creator only — the default for drafts and app-created data	—	—
Shared (domain)	usable across the owning domain	**the owner** of the artifact (and only the owner) files a promotion request	a **Builder or above of that domain** (Builder, Domain admin or Administrator)
Marketplace (certified)	discoverable and importable by other domains	a **Builder / Domain admin of the owning domain** — the domain vouches for its artifact	the **platform Administrator** — the platform accepts it

Approving **is** the action: on approve the platform executes the governed effect — for datasets, a physical publish; the tier flips only when it verifies — and writes the audit.

Nothing enters the governed store without **documentation + passing checks** — a transparency gate that turns green only when an artifact is documented and in the lineage graph. Throughout the product the creator-only scope is always labelled **"Personal"** (never "Mine" or "My"), and a domain is named directly — **" domain"**, not "My Domain".

2.3 Archive, delete and version history

Every artifact carries the same three lifecycle controls, surfaced consistently on each tab — agent systems, apps, big bets, dashboards, knowledge workflows, files, datasets/metrics/knowledge docs. Only the artifact's owner (or an in-domain Admin) may use them, exactly the same authority that governs editing it; a viewer sees the history but cannot change it.

- **Archive** — a reversible *soft-hide*. The artifact leaves the working lists (a running agent is also stopped) but nothing is lost; **Restore** brings it back unchanged. Use "Show archived" to see and restore archived items. Archiving a Shared/Marketplace artifact hides it for everyone until restored.
- **Delete** — a confirmed, permanent removal of the artifact **and** its version history. It is guarded by an explicit "Confirm delete" step.
- **Version history** — every meaningful edit first **snapshots the prior state** as a numbered version (with author + timestamp). Open **History** on any artifact to see the timeline and **Restore** any earlier version. Restore is itself auditable and reversible: it snapshots the *current* state as a new version before rolling back, so you can always undo a restore. History is durable — it is mirrored to OpenSearch (see *Durable state — every store mirrors, one core*), so it survives redeploys, and it is purged only when the artifact is deleted.

One reusable helper (`lib/versioning.ts`) provides this to every store, so the behaviour — snapshot on edit, list, restore-creates-a-new-version — is identical wherever you are.

2.4 Four roles, assigned per domain

ROLE	WHAT THEY DO
Creator	the base role — creates and runs their **own** artifacts (datasets, workflows, agents, apps — Personal by default) and consumes anything shared or certified. Cannot promote, approve, or reach admin; files promotion requests.
Builder	the domain approver — everything a Creator can, plus review/approve domain promotions, deploys, knowledge and connections. An approver, **not** a people-admin.
Domain admin	everything a Builder can, plus administering the users of their **own domain(s) only** — invite, edit, reset credentials, deactivate/reactivate, and assign roles **up to Builder**. Never mints another Domain admin or an Administrator, and never reaches the Platform group.
Administrator	tenant-wide — users (the only role that appoints **Domain admins**), policy, certification to the **Marketplace**, cost caps; runs Admin (Platform)

The ladder is exactly **creator < builder < domain_admin < admin**. Earlier releases had two more roles — *participant* (view-only) and *agentic-leader* — both are **removed**: agentic-leaders migrated to **Creator**, and any legacy or unknown role normalises to Creator; nobody is ever auto-promoted to Domain admin. Roles are assigned **per domain** and **compiled to OPA**, so a role change takes effect everywhere at once.

2.5 The governance spine

One **gateway** (every model and tool call, with cost caps), one **policy engine** (OPA, deciding `allow` / `deny` / `requires_approval`), one **trace** (Langfuse), one **audit**. Two layers of policy stack: **tenant guardrails** that Administrators set and domains cannot override (default-deny egress, no plaintext secrets to agents, no cross-domain data without a grant, a model allowlist), and **domain policy** that Builders set within them. High-stakes actions don't fail silently — they queue as a **card** in the **Governance** inbox, where *approving is the action*: on approve, the platform executes the governed effect and writes the audit.

Three planes stay deliberately separate, and cross-link rather than duplicate:

- **Admin** (the "Platform Admin" control room, labelled **Admin** in the sidebar's Platform section) — *configures* the tenant (identity, models, egress, structure).
- **Governance** — *decides and records* (approvals, policy, audit, caps, access).
- **Monitoring** — *observes the artifacts* (runs, spend, traces, drift). **Components** watches the infrastructure (service + cluster health). And **Dashboards** watch the business KPIs.

2.6 Every assistant acts, not just chats

Each tab's built-in assistant is **agentic**, not a chat box. It runs one loop: **PLAN** with the reasoning tier, then **ACT** in a bounded tool-calling loop with the execution tier — calling that tab's governed tools, which are the *same* OPA-authorized, Langfuse-traced functions the UI uses (never a privileged path) — then **verifies**, and for anything side-effectful **stops at a human gate**. The Software build assistant is the fullest example: it scaffolds a repo, writes and commits code, previews the app, and assembles a **deploy review card** through `lib/software` — it never self-approves a go-live. Which models fill the two phases is set once at the gateway (see *Models & the LLM gateway*).

2.7 The Sovereign OS Assistant — one assistant, everywhere, governed

Above the per-tab helpers sits **one overarching assistant**, present on **every** tab (the **Ask the OS** launcher in the bottom-right opens it). It is **context-aware of the tab you are on** — it is grounded in that tab's context and surfaces that tab's tools first — while carrying the **whole-OS** picture: it can reach **every** governed tool across every tab, so it helps with the work in front of you *and* anything else in the platform.

Crucially, it is a **client of the OS's own MCP server**. It gets all information and takes all actions by dispatching through the **exact same governed path** (`handleRpc`) that external MCP clients like Claude Desktop use — under **your** delegated identity. So it inherits **identical guardrails**: the role floor, approval gates, OPA/RLS, and Langfuse audit all apply unchanged. There is **no privileged side-channel** — the assistant cannot do anything you could not do yourself through the governed tools.

It runs on the **one assistant model** (`sovereign-default`, routed through the governed LiteLLM gateway to STACKIT) using the same **PLAN → ACT** harness as the tab helpers. The panel is **transparent and honest**: each answer lists the governed tools it invoked, and if governance **blocks** a call (wrong role, or an approval is required) it says so plainly and flags the tool — it never pretends it acted. If the assistant model is unreachable, it shows the honest error.

Under the hood: `components/OsAssistant.tsx` (shell-mounted, tab-aware via the route) → `POST /api/assistant/chat` → `lib/assistant/agent-loop.ts` (`runOsAssistant`), which reuses the shared harness and executes every tool via `handleRpc`.

3 Getting started

3.1 What you need

- A container runtime + the `docker` CLI (Docker Desktop or Colima), **running**.
- `kind`, `helm`, and `kubectl` on your `PATH`.
- About **14 GB RAM / 6 CPU** free for the runtime VM (the slice is RAM-bound).

3.2 Install in one command

```
# prereqs: docker (running), kind, helm, kubectl
./install.sh            # press Enter through every prompt
```

Pressing **Enter** through every prompt gives the **fully self-contained** install: every backend is bundled and a small local model answers model calls — nothing external is required. `install.sh` creates the `kind` cluster if needed, bootstraps the operators, builds and loads the images, installs the chart, seeds the demo data, and prints the **front door** and **demo logins**.

```
./install.sh --defaults     # non-interactive, all bundled (CI / quick)
./install.sh --uninstall    # remove the release (keeps the cluster)
```

The wizard asks, in order: the **target cluster** (`kind` / `stackit` / `other`); per-backend **bundled or external** for Postgres, OpenSearch, object storage and cache; and the **LLM endpoint** (local mock, or a real provider whose key is stored as a Kubernetes secret reference, never inline). Anything other than *bundled* is written into a small generated overlay as `mode: external`.

3.3 Open the front door

Everything is reachable by port-forward. Start with the OS UI:

```
# OS UI — the product front door (Home, Cockpit, Strategy, … , Governance)
kubectl -n agentic-os port-forward svc/os-ui 8080:3000          # http://localhost:8080
```

The **OS UI** is a Next.js app with a left sidebar. Every surface calls the in-cluster backends through **server-side API routes**, so credentials and keys never reach the browser. The stack's operational console is **embedded inside the OS UI** at **Platform → Components** — it reads the in-cluster Kubernetes API and the baked-in component docs natively, so there is no separate admin-console service to port-forward. Locally there is no login; on a real deployment you sign in with your Ory identity.

On a real deployment the back-end tools no longer need their own port-forward or login: they load **same-origin, inside the OS UI**, proxied by the Node server at `/tools/<tool>`. The server turns your OS session into whatever the upstream tool needs — **Level-1 header SSO** injects your identity (`X-Forwarded-User`, plus a tool-specific alias like Forgejo's `X-WEBAUTH-USER`) mapped through your per-

domain role, and the tool **auto-provisions** the matching account on first request. Credentials never reach the browser, and the proxy pins `frame-ancestors 'self'` so only the OS shell can embed the tool. (The port-forward table below is the local-dev path.)

The handful of back-end consoles you may want first (the full table is in the Reference):

CONSOLE	PORT-FORWARD (`KUBECTL -N AGENTIC-OS …`)	URL	LOGIN
OS UI	`port-forward svc/os-ui 8080:3000`	`http://localhost:8080`	— (local)
Langfuse (traces)	`port-forward svc/agentic-os-langfuse-web 3000:3000`	`http://localhost:3000`	`admin@datamasterclass.com` / `langfuse-local-dev-admin`
Superset (BI)	`port-forward svc/agentic-os-superset 8088:8088`	`http://localhost:8088`	`admin` / `superset-admin-local-dev`
Forgejo (git)	`port-forward svc/forgejo-http 3001:3000`	`http://localhost:3001`	`gitea_admin` / `forgejo-admin-local-dev`

> **These are local dev throwaways** (profile `local`), clearly marked as such and never reused on STACKIT, where every secret is external. See the *Security model* chapter.

3.4 Bootstrap: the first administrator, then real users and roles

On a real deployment, the platform starts **closed**. Getting from an empty install to a working team is a deliberate, audited sequence — this is the secure first-run path:

1. **Claim the first administrator.** Identity is backed by **Ory**; the secure first-run bootstrap creates exactly one tenant **Administrator**. This account **auto-verifies** — no email server is required to start using the platform — and no password is ever shown in the UI; the Admin sets credentials through the identity flow.
2. **(Optional) Wire up email** for later verifications and invites. The platform sends through one of two transports (precedence **Graph → SMTP → none**), sender `support@datamasterclass.com` by default:
   - **Microsoft Graph** `sendMail` (recommended for Microsoft 365) — an **Entra app registration** with the Graph **Application** permission `Mail.Send` (+ admin consent), called via OAuth2 client-credentials. This avoids SMTP basic-auth, which M365 is deprecating.
   - **SMTP fallback** — a minimal built-in client; set `SMTP_HOST` (+ port / user / pass / secure). Use this for any generic mail relay. With neither configured the platform simply runs without email; the bootstrap admin already auto-verified, and later accounts can be verified out of band.

> **Honest status on the live STACKIT tenant:** outbound mail is currently **not delivering**. An in-cluster sovereign sender (maddy, DKIM-signing) is built, but STACKIT blocks outbound port 25 and the smarthost relay + sender-domain DNS records are still pending — so on that deployment accounts are verified out of band. The Graph and SMTP transports above work wherever those services are reachable.

3. **Open Admin.** As that Administrator, go to **Platform → Admin** (`/platform`, hard-gated to admins). The **Overview** cockpit shows component health, spend versus the tenant envelope, users and domains.
4. **Create your domains.** In **Admin → Domains**, create a domain per team or business area, and toggle its optional layers (for example, turn Science on for a domain that needs ML).
5. **Invite real users.** In **Admin → Users & Access** (or **Governance → Users & access**), invite people **by email — the email is the username** — pick their **domain memberships** (a multi-select), and give each a **role per domain**; the role picker explains what each role can do. The platform generates a **one-time temporary password** that the Administrator copies and shares out-of-band; the invitee must set their own password on first login. If a mailer is configured the invitee also receives a verification email.
6. **Roles take effect everywhere.** Each assignment **compiles to OPA**, so a person who is a *Builder* in one domain and a *Creator* in another sees exactly the right controls in every tab, immediately.
7. **Set the guardrails.** Still in Admin, set the **model allowlist and defaults**, the **egress allowlist**, and the **cost envelope**. These compile through the same policy engine the tabs enforce — configuring a right here is never a governance bypass.

From here, each person opens the **OS UI**, lands on **Home**, and follows the golden path for the work they need to do — the rest of this guide.

3.5 Try the seeded demos

Four end-to-end demos ship seeded, so the system proves itself the moment it is up:

```
# 1. Ask the RAG agent — retrieve (OpenSearch) → generate (LiteLLM) → trace (Langfuse):
kubectl -n agentic-os run ask --rm -i --restart=Never --image=curlimages/curl:8.11.1 -- \
  curl -sS http://sample-agent:8000/ask -G \
  --data-urlencode "q=What provides the retrieval backbone?"
```

The reply includes the answer, the retrieved knowledge titles, and `traced_in_langfuse: true` — open Langfuse to see the run. The other three: **query the lakehouse** (the governed `query` tool over central Trino), **build a dashboard** in Superset on the seeded `daily_revenue` dataset, and **ship software** (push to the seeded `demo-app` repo → Forgejo CI builds an image → Argo CD redeploys). Each has a one-card launcher on **Home**.

4 Using the platform, tab by tab

Every chapter below follows the same shape: **What it's for • The golden path • Roles • Connects to**. They are in **sidebar order** — the business tabs first, then a short **Platform** section for the operational consoles. Most flows run live against the cluster and fall back to a labelled offline mock on `kind` — a ✓ always means a real apply and verify.

Every page also carries a **top-left ActionBar** under its title: a **Tutorial** button on every tab that has a golden-path tutorial, and — on the tabs that expose an MCP endpoint — a **"Connect your AI Tool via MCP"** button that opens a drawer with your per-user token and one-click import instructions for Claude/ChatGPT (see *Use the OS from Claude or ChatGPT* in the Reference).

4.1 Home — the golden-path launcher

What it's for. The warm front door after you pick a domain. Home **only orients and routes**: an illustrated launcher of the ten golden paths, with copy and dimming that shift by persona. The live "what's moving / what needs me" view now lives one click away in the **Cockpit** — Home no longer carries the cockpit modules.

The golden path.

1. Land on the editorial hero with your name, **domain**, and persona badge.
2. Scan the **Golden paths** launcher — ten illustrated cards (Data, Knowledge, Agents, Software, Science, Metrics, Dashboards, Big Bets, Marketplace, Connections), each with a role-aware primary action. Cards your role can't act on yet are explained but dimmed.
3. Click a card's action to deep-link straight into that tab's flow — or its tutorial link to learn it in place.
4. Use the **"Your cockpit"** call-to-action to open the live cockpit when you want to see what's moving and what needs you.

Roles. Every role — the launcher copy and dimming reorder by persona. **Connects to.** Routes into all ten path tabs; hands off to **Cockpit** for the live view.

4.2 Cockpit — what's moving, what needs you

What it's for. The live, governed overview that used to sit under Home. A persona-ordered cockpit of your work: a headline pulse strip, the working modules, and a scannable **top-items-per-artifact** board. Cockpit *reads and routes* — it never recomputes another tab's numbers and never bypasses governance.

The golden path.

1. Read the hero greeting and the **pulse strip**: *Needs you • In progress • Your items • Spend* vs. cap, each linking into its owning tab.
2. Work the persona-ordered modules under **"What's moving"** — *What needs me • My WIP • Domain pulse • Health & cost • Recent activity • Ask anything* — reordered to your role.
3. Clear an item from **What needs me** (an approval or draft) or pick up **My WIP**.

4. Scroll to **"Top items, by type"** — the most-notable thing you can see in each part of the registry, governed and scoped (never another domain's, never someone else's drafts).
5. Use **Ask anything** to get an answer or scaffold a Personal draft (promotion stays human and traced).

Roles. Every role — the modules and ordering shift by persona; everything is OPA/RLS-scoped.
Connects to. Reads Governance, the artifact registry, Strategy and Monitoring; routes into every tab. Domain pulse and spend feeds run live on a deployed cluster; on a local `kind` install they fall back to labelled offline stubs.

4.3 Strategy — pillars, value and adoption

What it's for. Where the company plans its agentic transformation, in exactly **three sections**, top to bottom. Calm and Apple-grade; governance stays server-side.

The three sections.

1. **Big Bets** — your strategic **pillars**, shown **side by side**, each holding the big bets that deliver its business value. Each pillar shows its current value + metric, its bets (with ready / in-progress / planned counts) and inline create/edit. A bet box opens the bet detail (below).
2. **Self Service** — how broadly your people build for themselves: **Total Users • Analytics • AI • Software • Builders • Creators** — distinct creators by capability area plus the builder/creator population, scoped to the viewer's company/domain.
3. **Foundations** — the governed asset base: **promoted + certified** artifact counts **per type**, the certified backbone every bet builds on.

> The old pillars/targets/value-rollup boxes and the RLS explainer are **gone**. A pillar's **value metric** is described here in business terms, then kept one of two ways: **set up as a governed Cube Metric** (hand off to Metrics) or **tracked manually each month** (a small value entry feeds the pillar's history chart).

The bet detail (opened from a pillar's bet box): a top row of **Value** + Planned / In-progress / Ready counts, then **Value metric over time → Roadmap** (each component on its due-date timeline, with a go-live marker) **→ Components** (each one a box that deep-links to *Edit in*) **→ Audit** (a subtle governance footer). There is **no Composition** view.

Roles. Administrators define **company** (tenant) pillars; Builders define **domain** pillars; Creators view.
Connects to. Big Bets (each bet links up to a pillar), **Metrics** (the governed Cube metric a pillar can track), every Build tab (the foundation counts), **Monitoring**.

4.4 Big Bets — initiative roadmaps

What it's for. A strategic AI bet as a **goal + dated roadmap** built from real artifacts across the platform, linked up to a Strategy pillar with a value target.

The golden path.

1. Browse the **Portfolio**, **grouped by strategic pillar** — each pillar is its own section of bet cards (realized vs. target value, a completion bar, component count, go-live date, at-risk flag).

2. **New Big Bet** opens a drawer whose fields are, in order: **Owner → Strategic Pillar → Problem Statement** (required) **→ Solution Idea → Value** (the chosen pillar's metric + currency, with a target) **→ Planned Go-Live**. Choosing the pillar carries its metric onto the value field.
3. Open the bet → its detail is **Value → Roadmap → Components → Audit**, with **Archive** — and **no Composition**. Add **components** by linking existing artifacts or scaffolding new ones through each tab's governed flow (the planner proposes a breakdown but never ships).
4. Watch **status derive live** from each artifact's real lifecycle (planned / in-progress / ready / blocked); the roadmap rolls up on-track / at-risk and flags an unrealistic go-live.

Roles. Anyone can draft a bet; advancing and promotion stay human (Builder/Admin); the planner runs as a non-promoting actor. **Connects to. Strategy** (pillar up-link, with a live Strategy↔Big Bets linkage), every Build tab (cross-tab artifact links and real metric values are live on a deployed cluster), **Monitoring** (runtime health). On a local `kind` install cross-tab component sources fall back to labelled offline stubs.

4.5 Agents — compose, govern, run

What it's for. One page where a domain's **agent systems** (instructions + tools + memory) are composed, governed and run, plus a strip of the **deployed agent systems** currently running. There are no sub-tabs. Every model and tool call goes through the gateway — no agent ever touches a raw resource. An agent system's tool calls dispatch through the **same governed MCP toolset** an external client uses, under its **owner's identity** — so an agent can never see or do more than its owner, and a `requires_approval` effect queues to Governance instead of executing.

The golden path. The tab is a **master–detail** surface: a rail of your agent systems on the left, the selected system's full detail on the right.

1. Author an agent system three equivalent ways — a **React-Flow drag-and-drop graph builder** (drag agents onto the canvas, connect the edges), **Monaco** editing of `system.yaml`, or the **agent-system assistant** chat — all editing the same versioned file. Each agent's `AGENT.md` and `MEMORY.md` open and persist reliably alongside the graph.
2. Grant the **resources** (data products, knowledge, files, connections) and **tools** the system may use; a validation gate must pass first.
3. Pick models with the single **Auto / Reasoning / Execution** toggle — it shows the **real gateway model names** (`sovereign-reasoning`, `sovereign-default`, …) with an **internal/external** badge per model; *Auto* routes each activity to the right tier, and a per-agent override writes the system's LiteLLM routing.
4. Press **Build** — *Build = execute + verify*: it runs the compiled system and checks it, every call routed through **LiteLLM → OPA → Langfuse**.
5. **Run / schedule / toggle** the system, or **fork-to-own** a copy. **Scheduling** provisions a real Kubernetes **CronJob** for the system (live on a deployed cluster). By default agents are *propose-don't-commit* — a write pauses for approval and enqueues in **Governance**.
6. **Promote** the system up the sharing ladder via the promote UX on the system card, then **certify** a finished system to list it in the Marketplace.

> **What is no longer here.** The Agents tab shows only **user-authored agent systems**. The platform's backend service agents — the **Domain RAG agent**, the **ML pipeline agent** and the **Hermes autonomous runtime** — are *system agents* now surfaced with live health on the **Platform** tab, not mixed into your authoring list; the old poet demo agent is removed entirely. When you select the Hermes runtime, an inline guidance strip explains exactly what **Run** will do in the current environment.

Roles. Creators and Builders author; promotion and held write-backs are human-gated. **Connects to. Knowledge** (attach-as-context, scaffold-from-workflow — both live), **Connections** (a connection becomes a tool), **Governance**, **Monitoring**, **Marketplace**. *(Build/Run execute against the live agent-runtime when a cluster is reachable, else an in-process mock — labelled either way.)*

Two runtimes, one governed plane. Each system picks a **runtime** next to the safety preset: **LangGraph** (the default — structured, replayable, human-in-the-loop graphs) or the autonomous **Hermes** runtime for long-running work that compounds — **persistent memory + self-improving skills**, running unattended. They are complementary, not competing, and share **one governed plane**: Hermes reaches models **only through LiteLLM** (direct provider keys disabled, so it cannot call a model off-gateway) and tools **only through the same Platform MCP** every other client uses — so **OPA still gates every call** (allow / deny / requires-approval), Langfuse traces it, and RLS scopes it to the profile's identity. There is no side door.

- **Model tier.** Hermes uses **Hermes 4.3** as its tool-calling brain, served via **vLLM behind LiteLLM** — the 14B tier is CPU-feasible; the stronger 36B/70B tiers sit behind the optional GPU pool. Magistral stays the general-reasoning default. *(Hermes 4 weights are Llama-3.1-based → Llama 3.1 Community License; fine for self-hosted use, not redistributed.)*
- **Safety presets** map straight to the profile: **read-only** (reads only, no unattended writes), **read + propose** (writes drafted for a human), **read + bounded writes** (bounded writes auto-run, approval-writes queue), **full in-scope** (everything the profile exposes, still OPA-gated). Out-of-scope tool calls are **OPA-denied and queued to the Governance inbox** — never a silent bypass.
- **Sandbox.** Code runs under a **real kernel-isolated runtime** — a **Kata microVM** where the node has nested KVM (an SKE preflight decides), else **gVisor** — **never** host-local. A blocklisted egress or a hardline command is refused (SSRF fail-closed, egress allowlist, website blocklist).
- **Skills & memory.** A skill Hermes creates surfaces as a **reviewable, uncertified artifact** (owner / domain / visibility) — the human promotion ladder still applies; nothing auto-certifies. Memory + skills persist to a **per-user, backed-up, deletable** volume (GDPR erasure honoured).
- **Gated off by default.** Hermes ships in the release but is **off in base and kind** (`hermes.enabled`), prepared for SKE — the runtime option is shown in the Agent tab regardless; the gateway provisions only where it is turned on. Hermes⇄LangGraph interop and messaging front-ends are later phases.

4.6 Software — build governed apps, sovereign

What it's for. One page: a big home-style **"Create new software app"** launcher, then your **running apps** as clean tiles. Git is **in-cluster Forgejo (sovereign)** — a repo is **auto-created in-cluster** on create; there are **no GitHub, no accounts, no tokens**, and your code never leaves.

The golden path.

1. Click **Create new software app**: name it and pick a template (web app / service / script / dashboard). It auto-provisions a sovereign **Forgejo repo in-cluster** and drops you straight into the app's **build chat beside a self-hosted Monaco editor** (`/software/{id}?mode=edit`).
2. Iterate in the **Claude-style build chat**: describe what you want; the agent writes the code and commits to the app's own repo, and you can edit files directly in the code editor.
3. Open any running app's tile → its **app page** offers two modes:
   - **Monitor** — live status, **"Open app UI"** / **"Show API details"**, and **Publish**.
   - **Edit** — the build chat + code editor again.
4. **Request deploy** assembles a **review card**: the security scan (SAST • dependencies • secret-scan), the requested **envelope** of governed resources, the cost/resource footprint, and the change diff. It enqueues to Governance under **Deploy reviews**.
5. A **Builder/Admin in the app's domain** decides it; a leaked secret or a high/critical finding **blocks go-live**, and a creator cannot self-approve. Routine **in-envelope** updates auto-deploy; anything that broadens scope re-opens the review.

Committing the app **auto-creates its MCP** from its OpenAPI spec (**reads on, writes held for approval**), governed by the same OPA gate every connection uses — so the app is instantly usable as a tool by your agents.

The in-cluster runner (live preview URLs). Preview and go-live provision a **real running workload**, not a mock. `lib/software/runner.ts` creates a Kubernetes **Deployment** (1 replica, CPU/memory requests + limits from the app's footprint, a TCP readiness probe), a **Service**, and an **Ingress** into a dedicated runner namespace (`SOFTWARE_RUNNER_NAMESPACE` , default `agentic-apps` , auto-created if missing). The app is served on its **per-app host** — the app's `subdomain` , `<slug>.<domain>.<appsDomain>` — with a cert-manager TLS cert issued by the same cluster-issuer and ingress class the platform consoles use (`OS_APPS_INGRESS_CLASS` / `OS_APPS_TLS_ISSUER`). The image is the app's **CI-published registry artifact** (`<registry>/<slug>:latest`) by default, or an explicit prebuilt `runImage` , or the platform-wide `SOFTWARE_RUNNER_IMAGE` teaching placeholder — the OS **never builds images in-cluster**. Lifecycle status is **pod-driven**: `deploying → running → failed` comes straight from the Deployment's `readyReplicas` /conditions (poll `GET /api/apps/{id}/deploy`), and the **live URL only appears once the pod is actually running** — when no cluster is reachable the runner degrades **honestly** (no fabricated URL, status `offline`). **Archive** scales the runner to zero (objects retained, restartable); **delete** tears down the Ingress + Service + Deployment before removing the record, so nothing is orphaned.

The Software Delivery Team. Beside the solo build chat, the tab offers a full **six-agent LangGraph system** that takes a brief through the whole lifecycle: an **orchestrator** (delivery lead) routes the work through **planner → builder → tester → deployer**, and a **communication** agent reports back. Model routing matches the two gateway tiers — the **builder** codes on `sovereign-default` (the execution tier); the other five reason on `sovereign-reasoning` . A **per-user graph executor** runs every node's tool calls **as the signed-in user** — the team's grants are exactly the Software tab's governed write surface (create, commit, preview, request deploy), and `decide_deploy` is deliberately **not** granted, so the team can assemble a deploy review but a **human Builder** still decides the go-live. The same `system.yaml` is seeded as a domain-**Shared** system in the Agents tab, so every Creator can *run* the team (never edit it).

Roles. Any owner previews; a Builder/Admin-in-domain approves go-live; an Administrator certifies to the Marketplace. **Connects to. Connections** (the auto-MCP), **Governance** (deploy reviews), **Data** ("Use as Data"), **Marketplace**, **Monitoring**.

4.7 Science — classic ML (opt-in, Layer 4)

What it's for. Taking **traditional ML** (regression, classification, forecasting, clustering — *not* LLMs) from a governed data product to a governed, deployed **model-as-service**. Off by default; GPU is optional and cost-gated.

The golden path.

1. If the tab is off, a disabled surface explains that an **Administrator** enables ML per domain (`ML_ENABLED=true`).
2. When on, check the **Layer-4 stack** health grid (Featureform, MLflow, Dagster, KServe, JupyterHub).
3. Walk the guided path on the seeded slice: **Explore → Features → Train → Register → Certify → Deploy → Consume → Monitor** (an ML agent drives the no-code common path; JupyterHub is the escape hatch).
4. Move up the **tier ladder** (Personal → Shared → Marketplace): a **Builder** promotes and go-lives Staging → Production; an **Administrator** certifies, choosing read-in-place or fork-allowed.
5. A deployed model exposes **two front doors from one KServe endpoint** — a governed REST `predict` API and a governed `predict` MCP tool — both through OPA + LiteLLM, capped and traced.
6. Watch **drift** (PSI vs. AUC). *Honest status:* the drift series fills from live MLflow/KServe telemetry once a model is serving — a fresh tenant shows an empty panel, never fabricated history — and the **retrain** trigger is a **v1 scaffold**: it stages the Dagster job reference, but the retrain pipeline itself is not wired yet. The ML agent proposes only — it is hard-blocked from certify and go-live.

Roles. Administrators enable + certify; Builders promote/go-live; Creators build/train; the agent proposes. **Connects to. Data** (the governed mart), **Software**/external (REST), **Agents** (MCP), **Monitoring** (the same drift signals), **Marketplace**.

4.8 Knowledge — the domain's operating manual

What it's for. The domain's human-authored manual: general domain knowledge plus a **workflow** per business process, made retrievable by a knowledge agent behind document-level security.

The golden path.

1. In **Domain knowledge**, edit the four guided sections (overview / glossary / goals / context) — auto-pinned as the base context for every domain agent. The **knowledge agent** can draft; you paste the result in.
2. Switch to **Workflows** and browse tiles grouped by **visibility** — **Personal** (your own drafts), **Domain** (Shared workflows the whole domain sees), and **Marketplace**. Each tile offers **Archive** (a reversible soft-hide); **Show archived** reveals archived workflows with **Restore** or permanent **Delete**.
3. **+ New workflow** → name a business process → open the detail editor: an **actor-coloured swimlane** whose lanes (Human / Software / Agent) are **vertical columns and flow top → bottom**, **Monaco markdown**, and a derived diagram — all editing one versioned `workflow.md`.

4. Capture per-step actors, inputs/outputs and **links** (a gap jumps you to where to build it); mark a decision rule **hard** to compile it into an **OPA guardrail**.
5. A **Builder/Admin publishes** a draft to make it live; **certify** lists it as a knowledge product.
6. Hand a certified workflow off to **scaffold a domain agent**.

Roles. Creators author Personal drafts; Builders/Admins publish and certify; document-level security keeps Personal units private. **Connects to. Files** ("Use as → Knowledge"), **Agents** (context pack + agent scaffold), **Marketplace**, **Governance**.

4.9 Files — a calm governed drive

What it's for. A drive for any **unstructured** file — documents, images, video, audio, archives — auto-indexed so agents can search and cite it, governed exactly like Data.

The golden path.

1. In **Files**, **upload** anything (drag-drop, any type) and watch the status chip move *Processing* → ***Searchable*** ✓. Behind it: parse (Docling for docs, transcription for A/V, OCR/caption for images) → embed → hybrid OpenSearch index.
2. Organize with **folders + tags** and preview in place.
3. **Search** across names, tags and content (DLS-filtered to what you may see).
4. In **Sources**, connect **Google Drive / OneDrive** via OAuth — the OAuth app for each must be registered once by a platform Administrator in **Admin → Connections** (see that chapter); once registered, any user can connect their own account. Synced files **re-govern** under our tiers, not the source's ACLs.
5. **Promote/certify** up the ladder (a light docs gate: owner + description + at least one tag); `restricted` files are stored but never indexed.
6. **"Use as"** distils a file into **Knowledge** (a tacit note) or **Data** (a guided Bronze import), with lineage recorded.

Roles. Reuses Data's gates — Creators upload Personal; Builders promote; Administrators certify. **Connects to. Knowledge** and **Data** (Use-as), **Connections** (Drive/OneDrive), agents (the `files_retrieve` tool), **Marketplace**, **Governance**.

4.10 Data — datasets, refined and governed

What it's for. Turning a plain-language flow — refine a dataset **Bronze → Silver → Gold**, then share it — into real governed artifacts (a dlt pipeline, dbt models, a Cube cube), with no YAML for non-technical users. The tiles are grouped **Data** (your Personal datasets) • **Shared Data** (domain) • **Marketplace Data** (certified, cross-domain).

The golden path.

1. Open the **Datasets** tab and create or open a dataset; it presents as guided **Bronze / Silver / Gold** panels — one logical dataset in three versions.
    - **Bronze** — bring it in: upload a file or pull a masked slice of a product.
    - **Silver** — clean it up: fix types, drop duplicates, set the key.

    - **Gold** — make it ready: harmonize to the shared shape and add quality checks.
2. Press **Build** on a stage — it runs that stage's adapters (apply **+ verify**); the row turns ✓ only when both pass. Toggle **"Show the code"** to see the real artifact. On a live cluster the chain is **physical end to end**: an upload lands as a real Iceberg table in your **own per-user schema** (`iceberg.personal_<you>.bronze_…`), Silver and the Gold **join** build real tables the same way, and everything is queried through governed Trino under your identity.
3. Personal work stays governed per-user: your Bronze/Silver/Gold tables live in your per-user Iceberg schema, and the separate Query **sandbox lane** (DuckDB) sits *behind* the same Trino governance boundary — it only ever sees your own uploads or an already-masked extract. Datasets that have not been materialized yet are shown with an honest **not-materialized** state label — the tab never fabricates a green ✓ for a stage that has not been built.
4. Browse structured assets in **Catalog** (OpenMetadata); **row-preview** any dataset inline (a governed sample, DLS-filtered to what you may see) and **Preview** any into Query.
5. Ask questions in **Talk to your data** — governed **NL→SQL**: the model is shown only the datasets *you* can see, generates exactly one read-only SELECT (validated before it runs), executes through governed Trino under your row filters and masks, and answers grounded only in the returned rows — or run SQL yourself in **Query**.
6. Share like a review, on the sharing ladder: the **owner** requests promotion → a **Builder+ of the domain approves**, and the approval **runs the physical publish** (the tier flips only when it verifies) → certification to **Marketplace Data** is filed by the domain (Builder/Domain admin) and approved by an **Administrator**.

Roles. Creators create datasets; owners request promotion; Builders (and Domain admins) approve to Shared Data; Administrators approve certification to Marketplace Data (approvals run through Governance). **Connects to. Metrics** (the Gold auto-cube), **Dashboards**, **Knowledge/Files** ("Use as"), **Marketplace**, **Governance**.

4.11 Metrics — one number, everywhere

What it's for. The KPI semantic layer. Define a measure once and "Revenue" or "Active customers" resolves to the **same number** in the explorer, in dashboards, and in the agent's `metrics` tool.

The golden path.

1. Open the **Registry** and browse every governed metric (tier-honest).
2. **Define** a measure three convergent ways — a friendly **form**, the **metrics agent** ("define revenue on sales"), or hand-edited **Cube YAML** — all producing the *same* canonical member (e.g. `Sales.revenue`).
3. **Explore** it: slice by dimensions with no SQL, under your own row-level security (two viewers see different rows); drop to SQL/Trino if you need to.
4. **Govern** it: promote Personal → Shared (Builder) → Marketplace (Admin); promotion runs a consistency check (documented, defined, resolves on its member).
5. Inspect quietly in **Live Cube**. The metric now feeds Dashboards and agents identically — and can be the governed value metric behind a Strategy pillar.

> **Live demo data (Northpeak Commerce).** The Metrics tab resolves real numbers on the seeded deployment: the Gold Iceberg table `iceberg.sales.gold_northpeak_commerce` — materialized from the Bronze/Silver pipeline by a Trino CTAS job — backs the `northpeakcommerce` Cube model. It exposes `revenue`, `aov` (average order value), `conversion_rate` and `churn_rate`, sliceable by `region`, `product` and `date`. Open **Live Cube** and slice without SQL to see it working end to end.

Roles. Creators define; Builders promote; Administrators certify. **Connects to. Data** (the base cube), **Dashboards** and **Agents** (consume the member), **Strategy** (pillar metrics), **Marketplace**.

4.12 Dashboards — governed BI

What it's for. Apache Superset dashboards built read-only on governed Cube metrics, so the BI layer and the agents can never disagree.

The golden path.

1. Open **Dashboards** — tiles grouped Personal / Shared / Marketplace.
2. Build one two convergent ways: **drag charts** in Superset (real Superset import, live), or ask the **dashboard agent** ("build me a Sales overview") — both edit the same dashboard, on the same metrics.
3. **Double-click** a tile to open it inline via the Superset Embedded SDK; a server-minted **guest token carries the viewer's RLS**, so a shared dashboard still shows only your rows.
4. Set a **threshold alert** on a metric → notify by email/Slack/in-app *and* optionally trigger a governed agent run (Langfuse-traced). Alert delivery is live.
5. Schedule a **report** (a dashboard snapshot on a cadence). Report delivery is live.
6. **Promote/certify** the dashboard up the ladder without ever broadening rows.

Roles. Creators build; Builders promote to Shared; Administrators certify to Marketplace. **Connects to. Metrics** (consumes members, never defines), **Agents** (alert-triggered runs), **Monitoring** (traces), **Marketplace**.

4.13 Connections — governed bridges to outside systems

What it's for. External systems only. A **Connection** is a governed bridge to an *outside* system — `credentials + endpoint + a set of governed tools`, never a raw pipe — used two ways: to **bring data in** (Database / API / SaaS → dlt → Bronze) and to **expose external APIs/MCPs as tools** for agents and software. You grant **use**, never the token: the secret stays in the secrets store. (Platform-service status moved out — that now lives in **Components**.)

Supported connectors. The Connections tab offers exactly **three** connectors today, each wired end-to-end so you connect your **own** account and only a token *reference* is stored (never a raw secret): **Google Drive**, **Microsoft OneDrive** (both personal read-only OAuth), and **Notion** (via Notion's **hosted MCP**, OAuth 2.1 • PKCE). There are no placeholder/"roadmap" connectors — you can never stand up a non-working mock connection from this surface.

The golden path.

1. In **My connections**, register a connection under the Personal → Shared → Certified lifecycle. Any user can connect their **own** account (Google Drive, OneDrive, or Notion) via per-user OAuth.
2. In **Registry**, see the three supported connectors and the auto-generated **App MCP connections** (one per Software app). Each connector's **Connect →** opens the Governed connections tab.
3. In **Governed connections**, set each tool's **capability profile** — a mode of *Off / Read / Write-approval / Write-bounded / Blocked*, plus scope, rate and cost limits. **Reads on, writes off by default**; the profile compiles to an OPA policy, and a grant to an agent can only *restrict*, never broaden.
4. In **Build a connector**, describe a source and the connections agent drafts the connector and the credentials it needs.
5. A write pauses for approval: **with a human present**, inline with a before/after preview; for **autonomous agents**, bounded by a Builder-set safety preset, with anything out of policy blocked and queued in Governance.
6. New egress endpoints are **Builder-request → Admin-approve**; all outbound traffic is logged.

Roles. Any user adds personal connections; Builders/Admins add shared credentials and attach connections to agents; Administrators manage the egress allowlist and Marketplace. **Connects to. Agents** (connection = tool), **Data/Files** (connection = source), **Software** (the auto-MCP), **Governance** (egress + write-back approvals), **Marketplace**.

> **v1 status — read this.** The three connectors in **Governed connections** (Google Drive, OneDrive, Notion) are wired end-to-end: real per-user OAuth, a durable token *reference* in Secrets Manager with silent refresh, a verified connected state, and — for Notion — a live MCP `tools/list` proof. The **Build a connector** agent still *drafts* a configuration for review rather than minting live credentials (marked *"scaffolded in v1"* on screen). Governed tool **execution** for a connected Notion workspace beyond `tools/list` is the next increment; the capability-profile, OPA policy and approval spine already govern every call.

4.13.1 CONNECTING GOOGLE DRIVE / ONEDRIVE

Any user can connect their **own** Google Drive or OneDrive with their **own** account — no admin in the loop per user. It is a two-step, least-privilege OAuth flow.

For each user (Connections → Governed connections):

1. Under **New connection**, pick **Google Drive (personal)** or **OneDrive (personal)**, give it a name, and click **Add**. This creates a private (Personal) connection with no live token yet — its card shows **Not connected**.
2. On the card, click **Connect**. The page navigates to the provider's consent screen, where you sign in as **yourself** and approve **read-only** access. You are sent back and the card now reads **Connected as** `<you>`. The token set is stored in Secrets Manager — never in the browser, the record, a log, or a trace; only a fingerprint is ever shown.
3. A stale token is refreshed silently. If a refresh ever fails, the card shows **Needs reconnect** — click **Reconnect** to re-consent. **Disconnect** removes the connection and its stored token.
4. If the provider's OAuth app has not been registered yet, the card says *"An administrator must configure the Google/Microsoft OAuth app first"* and **Connect** is unavailable until they do.

For the platform administrator (one-time setup). Register the tenant's Google and Azure OAuth apps once under **Platform → Drive OAuth apps**. See the operator note below for the exact steps. The client secret is written to Secrets Manager server-side; the catalog keeps only a reference + fingerprint and never shows or logs the raw secret.

> **Operator note — registering the Drive OAuth apps (one-time).**
>
> **Google Drive.** In **Google Cloud Console → APIs & Services → Credentials**, create an **OAuth client ID** of type *Web application*. Add the authorized redirect URI **exactly**: `https://agentic.datamasterclass.com/api/connections/oauth/google/callback`. On the OAuth consent screen add the scope `https://www.googleapis.com/auth/drive.readonly` (read-only). Copy the **client id** and **client secret** and paste them into **Platform → Drive OAuth apps → Google Drive**, then click **Register**.
>
> **OneDrive (Microsoft).** In **Azure Portal → App registrations**, create a registration (multi-tenant is fine — the connector uses the `common` endpoint). Under **Authentication**, add a **Web** redirect URI **exactly**: `https://agentic.datamasterclass.com/api/connections/oauth/microsoft/callback`. Under **API permissions**, add the Microsoft Graph **delegated** scopes `Files.Read` and `offline_access` (the latter yields a refresh token for silent renewal). Create a **client secret** under *Certificates & secrets*, then paste the **application (client) id** and the **secret value** into **Platform → Drive OAuth apps → OneDrive** and click **Register**. The optional **tenant** field can be left blank.
>
> The redirect URIs must match character-for-character. The raw secret is stored only in Secrets Manager; if you ever need to rotate it, register again — the panel replaces the reference + fingerprint.

4.13.2 CONNECTING NOTION (HOSTED MCP)

Notion connects via its **hosted MCP server** (`https://mcp.notion.com/mcp`) — the OS acts as an MCP *client* and you authorize your **own** Notion workspace. Unlike Drive/OneDrive there is **no admin setup**: the flow uses OAuth 2.1 with **dynamic client registration** and **PKCE**, so the OS registers itself with Notion on the fly.

For each user (Connections → Governed connections):

1. Under **New connection**, pick **Notion (personal • hosted MCP)**, give it a name, and click **Add Notion**. This creates a private (Personal) connection with no live token yet — the card shows **Not connected**.
2. On the card, click **Connect Notion**. The OS discovers Notion's OAuth endpoints (RFC 9728 → RFC 8414), registers a public client (RFC 7591), and navigates you to Notion's consent screen with a PKCE challenge. Sign in as **yourself** and choose which pages/workspace to share. You are sent back and the card reads **Connected as** `<you>`. The token set is stored in Secrets Manager — never in the browser, the record, a log, or a trace; only a fingerprint is shown.
3. Click **Verify • list tools** to *prove the connection is live*: the OS runs a real MCP `initialize` + `tools/list` round-trip through your stored token and lists the tools the Notion server exposes. A stale access token is refreshed silently first; if refresh fails the card shows **Needs reconnect**. **Disconnect** removes the connection and its stored token.

The egress allowlist already includes `mcp.notion.com` and `api.notion.com`. No client secret is required (public client + PKCE); if Notion ever issues one it is stored only in Secrets Manager.

4.14 Marketplace — consume across domains

What it's for. The *Consume* counterpart to every Build tab's **certify** step: discover and reuse **Administrator-certified products of every type** across the tenant's domains. Importing is a **governed grant**, not a copy.

The golden path.

1. Browse the cross-domain catalog; filter by **type**, domain or tag, or search.
2. Read each listing's **source pill** (live / offline-mock) and its certification badge, owner, lineage, quality and usage signals.
3. Open a listing → an RLS-filtered **preview/sample**, lineage and ratings.
4. **Import.** The default is **read-in-place**: you consume under **your own identity + row-level security**, and the owner's certified artifact stays the source of truth. Some types differ — an **app** deploys your own real running instance (live), a **connection template** creates a real governed Connection in your personal lane (bring your own credentials, then use immediately), an **agent** is fork-to-own.
5. Approval-required imports create a pending grant + a **request in Governance**; clearing it activates the grant.
6. Track grants in **My imports**, rate 1–5; Administrators **deprecate** lineage-aware (importers are warned; in-use grants are kept).

Roles. Certification is filed by the owning domain (Builder/Domain admin) and approved by an Administrator (upstream), who also deprecates; anyone discovers and imports. **Connects to.** Every Build tab's certify step, **Governance** (import approvals), **Monitoring**.

4.15 Monitoring — artifact observability

What it's for. The read/observe plane for your **artifacts**: trace runs, watch spend, and surface pipeline + model **drift** — scoped to your identity. It is strictly **read-only**; it does not set policy or caps (that's Governance), it does not watch business KPIs (that's Dashboards), and **infrastructure health lives in Platform → Components** — it is deliberately *not* a Monitoring lens.

The golden path.

1. Read the **scope pill** (your Ory→OPA identity).
2. Scan **Needs attention** — the few red/amber items that need a human lead, not a wall of green.
3. Review the **four lenses**: **agent & run** observability (Langfuse), **data-pipeline** health (Dagster + dbt), **cost & usage** vs. the Governance caps (LiteLLM), and **artifacts** across all tabs including ML (freshness, lineage, **drift**).
4. Click any run/pipeline/model → the **trace drawer**: the full Langfuse trace (steps, tool calls, the context pack, inputs/outputs) plus logs — gated to your scope.
5. Follow the **correlation spine** — run ↔ pipeline ↔ artifact — and cross-link to the Governance audit entry and the cost cap it spends against.

Roles. A Creator sees only their own runs/cost; a Builder their domain; an Administrator the tenant. A Creator cannot open another user's trace (enforced server-side). **Connects to.** Reads from every tab; reads Governance caps; defers KPI alerts to Dashboards, cap-setting to Governance, and infra health to Components.

4.16 Governance — the control plane

What it's for. Consolidate, decide, record. Governance *enforces* policy and *executes the effect behind a decision* — but it doesn't author tenant structure (that's Admin). Its sidebar tab sits at the **top of the Platform group**, visible from **Builder rank up** — Builders, **Domain admins** and Administrators, the people who approve. Creators don't need the tab: their own request status is shown **in context** (the Promote / Certify panels on the artifact itself).

The golden path.

1. Work the **Approvals inbox** — one scoped queue of every side-effectful, role-gated action: a Software deploy review, an autonomous out-of-policy action, an access/import request, a new egress endpoint, a promote/certify. **Approving is the action**: on approve the platform runs the effect (Argo deploy • policy grant • egress allowlist • promote • the queued run) and writes the audit.
2. Read the consolidated **Policies** plane; an Administrator can override (revoke a grant).
3. Search the hash-chained **Audit** (who / when / why) and verify chain integrity.
4. Set a **cap** in **Cost & limits** — over-cap is enforced.
5. Manage **Users & access** — invite **by email** (the email is the username), pick **domain memberships** in a multi-select, and assign a role — the picker describes each role. The platform generates a **one-time temporary password** the inviting admin copies and shares out-of-band; the invitee must change it on first login (a raw password is never stored or re-displayable). A **Domain admin** works this surface for their **own domain(s) only**: invite, edit, deactivate/reactivate, and assign roles **up to Builder** — never another Domain admin or an Administrator (only the platform Administrator appoints Domain admins). Existing users can be **edited**, and retired ones walk a safe lifecycle: **archive → restore → permanently delete**, each behind an explicit confirmation dialog; the last active admin can never be archived or deleted. Everything compiles to OPA.
6. Use **"Approve & remember"** to turn a decision into an editable standing policy.

Roles. Creators see and act on their own requests; Builders their domain's queues, policy and audit; Domain admins additionally their own domains' users and memberships; Administrators the whole tenant. **Connects to.** Software, Connections, Data/Files, Agents/Science and Marketplace all raise cards here; **Admin** compiles the identity Governance reflects; **Monitoring** watches.

4.17 The Platform section

The sidebar closes with a small **Platform** group of operational consoles. **Governance** is visible from Builder rank up; the remaining entries (**Admin**, **Components**, **Terminal**, **About / Licenses**) are Administrator-only.

4.17.1 ADMIN — THE TENANT CONTROL ROOM

What it's for. A tenant-scoped, **Administrator-only** area above the per-domain workspace, where the tenant's structure, identity, models, egress and posture are authored — all of which compile through to OPA. Labelled **Admin** in the sidebar (the conceptual "Platform Admin").

The golden path.

1. Open `/platform` (hard-gated to admins) → the **Overview** cockpit: component health, spend vs. envelope, users, domains, and the principals compiled to OPA.
2. Work the **open admin alerts**, then jump via the quick links.
3. **Domains** — create, rename, archive or transfer; toggle a domain's optional layers (this scales already-provisioned workloads 0↔1; the UI never provisions cloud resources).
4. **Users & Access** — invite by email (email = username; the credential is never returned), set domain memberships and roles; archive/restore/delete run behind confirmations.
5. **Models & Providers** — configure the platform's single **assistant LLM**: the endpoint and key for the STACKIT managed model (or any compatible provider) that powers the built-in artifact-building assistants across every tab. Provider keys are stored as a **reference + fingerprint**, never raw.
6. **Drive OAuth apps** — register the tenant's **Google Drive and OneDrive OAuth apps** once (client ID + secret for each; the secret is stored as a reference + fingerprint, never raw) so users can connect their **own** accounts from the **Connections** tab. See *Connecting Google Drive / OneDrive* above for the exact redirect URIs and scopes. **Security & Egress / Cost & Billing / Backups & Restore** — configure the remaining posture; all compile through OPA.
7. Destructive actions (restore, disable) require a **typed-confirmation guard** and are audited; identity/domain/egress/model changes **re-compile to OPA**.

Roles. Administrators only. **Connects to. Governance** (enforces and shows the compiled plane), **Components** + **Monitoring** (watch live health and spend), and every tab (via the OPA grants the compiler emits).

4.17.2 COMPONENTS — THE ONE OPERATOR SURFACE

What it's for. The operational console for the **stack itself**, embedded in the OS UI — and, after the nav consolidation, the **single** operator surface: the old **Gateway**, **Orchestration** and **Consoles** tabs folded into it (their routes redirect here). One calm list of **every platform service** with live health, **version**, and each service's actions on its own row.

The golden path.

1. Open **Platform → Components**; services are grouped **by layer** (infra, L1 core, L2 foundations, L3 self-service, L4 science, security & platform), each with a live status dot + version.
2. **Open a tool's console straight from its row** — same-origin at `/tools/<tool>` with Level-1 SSO where wired (Superset, OpenMetadata, Dagster via **Open Dagster**, Forgejo, MLflow, …), or the native console URL where configured (Langfuse, Argo CD, …). URLs come from the runtime env; a tool that isn't publicly exposed honestly shows no link. Dagster OSS ships no login of its own, so its public ingress sits behind an **operator basic-auth** gate (an htpasswd secret on the ingress).
3. **Expand a row** for the quiet details: address (port-forward + URL), login, docs — and on the **LiteLLM row**, the **model gateway diagnostics** (the model catalog + registered MCP tools that used to be the Gateway tab).
4. **Toggle** an optional workload on/off (scale 0↔1); core services are marked always-on. The routes read the in-cluster Kubernetes API and the baked-in component docs natively; the browser never touches a Kubernetes credential.

Roles. Administrators. **Connects to. Admin** (structure + layer toggles), **Monitoring** (artifact observability sits beside this infra view).

4.17.3 MCPS & APIS — THE GOVERNED TOOL-REGISTRY

What it's for. A read-only, **Administrator-scoped** registry of every MCP server and API the platform exposes or brokers — a page **inside Admin** (`/platform` → *MCPs & APIs*), divided into four groups.

1. **Sovereign OS MCPs/APIs.** The platform's own governed servers: the authenticated remote at `/api/mcp` (cross-tab, one per-user token) and per-tab servers at `/api/mcp/<tab>` (`software · data · science · knowledge · agents · files · metrics · dashboards · bigbets`), each scoping tools and a minimal `CONTEXT.md` to that tab's governed library functions. Every tool delegates to the same function the UI calls — OPA, role gates and Langfuse audit apply unchanged.
2. **Stack-tool MCPs.** Component-wrapped servers that ship with the platform — for example the LiteLLM MCP gateway and the governed `query` tool (central Trino over Iceberg, OPA-authorized).
3. **Shared app/connection MCPs.** Domain- and marketplace-tier MCPs: the auto-generated server minted from every certified Software app (from its OpenAPI spec), plus shared Connection MCPs.
4. **Personal (unshared) MCPs.** Personal Software apps and personal Connections not yet promoted.

Every entry shows a **"Import into Claude"** and **"Import into ChatGPT"** deep-link for one-click registration in an external client (live — the OAuth consent flow completes in the client on first use). The registry is **read-only here** — to govern tool capability profiles go to **Connections**; to promote an app's MCP go to **Software**.

Roles. Administrators only (full-tenant view); individual import links are surfaced in **Connections** and **Marketplace** for non-admins. **Connects to. Connections** (capability profiles), **Software** (auto-MCP per app), **Marketplace** (certified MCPs importable cross-domain).

> The remaining Platform entries are **Terminal** (the Admin developer surface — it **auto-connects on open** and **re-attaches to your live session** when you navigate away and back, so a running shell survives the trip) and **About / Licenses**. The former **Users**, **Gateway**, **Orchestration**, **Consoles** and **Workbench** tabs were consolidated: Users & Access lives in **Admin**, and the gateway, orchestrator and console launchers merged into **Components** (the tool UIs stay embedded **same-origin** at `/tools/<tool>` with Level-1 SSO); the old routes redirect. **Tutorials** moved up into the main tab group, just above Settings — it's for everyone, not only operators.

4.18 Tutorials — learn any path in place

What it's for. One illustrated, hands-on tutorial per golden path, authored once and reached two ways — Home's launcher and a tab header's **"Tutorial"** — that resolve the *same* registry entry, so they never drift.

The golden path.

1. Trigger a tutorial (a Home launcher card, or a tab header's "Tutorial").
2. Read the **Hook**, then 3–5 illustrated steps in the overlay (your route and scroll are restored on close).
3. Choose **"Walk me through it"** → live **coach-marks** spotlight the real controls, located by stable `data-tutorial-anchor` ids so the highlight can't desync.
4. **Practice** in sandbox mode on the tab's personal lane — every governed write is stripped, so nothing persists.

5. **Graduate** to do it for real, where OPA and RLS govern every step.
6. Finish on **"You did it"**, with cross-links to the next paths.

Roles. Framing is role-aware (Creator "create and run", Builder also "review/promote"); the core path is identical and OPA/RLS are never bypassed. **Connects to. Home** (the launcher gallery) and each path tab via its anchors. *(Anchors are wired incrementally — Data and Agents today; a tab without anchors degrades gracefully to an "open this tab" card.)*

5 Reference

5.1 Deploying to your cloud (STACKIT)

Locally everything is self-contained. On **STACKIT** (or any cloud) the platform runs the **full Layer 1–4 stack** — the **Data engine** (central Trino over the Iceberg lakehouse), the **metrics layer** (Cube), **Science / ML** (MLflow + KServe + Featureform + JupyterHub), **Orchestration** (Dagster), the developer **Terminal**, and sovereign git via in-cluster **Forgejo + Argo CD** — alongside L1 (LangGraph/LiteLLM/OpenSearch/Langfuse) and L2 (OPA/Docling/Haystack/dbt/ OpenMetadata). It is the **same chart**; switching a backend to a managed service is only a values choice (`values.stackit-managed.yaml`), and the heavier layers (Science, Terminal) ship **pinned and provisioned but off by default**, enabled per domain when needed.

1. **Prerequisites.** A STACKIT organization + project in **EU01 / Deutschland Süd**, and a service-account key with provisioning roles (SKE + Object Storage + DNS), saved as `stackit/sa-key.json` (gitignored). **This key is the gate for any live deploy** — you can build and validate the entire chart on local `kind` with no key.
2. **Provision the managed resources** (Terraform preferred): an **SKE cluster** (CNI = Cilium), a **node pool** sized for the full stack (≈3× g1.4 for L1+L2, 4–5× for L3, more if Science is on), **Object Storage** buckets + S3 credentials, a **load balancer + public IP**, a **DNS zone**, and **Secrets Manager / KMS**.
3. **Bootstrap the in-cluster platform** before the OS chart: ingress-nginx + cert-manager, the SKE storage class, Cilium default-deny egress, the External Secrets Operator, CloudNativePG, Velero, and Argo CD.
4. **Point the OS at managed backends** in `values.stackit-managed.yaml`: object storage and Postgres to STACKIT, the LLM to **STACKIT AI Model Serving** (`llm.mode: external`, `provider: stackit`), Trino → the Polaris REST catalog (OAuth2), plus ingress hostnames (Forgejo, Superset, the OS UI), the egress allowlist and per-domain quotas. You can mix freely — managed Postgres but bundled OpenSearch, for example — and turn Science / Terminal on per domain.
5. **Deploy and verify:**

   ```
   helm install agentic-os charts/sovereign-agentic-os -n agentic-os --create-namespace \
     -f values.stackit-managed.yaml -f values.generated.yaml
   ```

   Point DNS at the load balancer, confirm the consoles, confirm the default-deny egress baseline is active, then create your first domains.

Cost. Roughly **€450–670/mo** for L1+L2 at typical sizing; the full L3 + Science stack scales it up. **Scale the node pool to zero between sessions** (storage + IP persist at ~€16–20/mo). LLM token spend is separate and **capped in LiteLLM**.

5.2 Sizing & capacity

Three different resources get confused under one word, "size." They scale for different reasons, so it helps to keep them separate. The verified single-node deploy runs on a STACKIT `m3i.16` worker — **16 vCPU / 128 GB RAM** — and in practice RAM ran at only **~2–4%**: memory is plentiful. The resource that actually bit us was the **node disk**, and it bites for a reason that has nothing to do with how much data you have.

RESOURCE	THIS DEPLOY	HOLDS	HOW IT SCALES
Node RAM	128 GB (`m3i.16`, 16 vCPU)	Running pods — Trino's JVM heap, OpenSearch, the in-box model	With **concurrency / workload**, not data. Ran ~2–4% here.
Node disk	**200 GB** (was 80)	**Container images + local model weights** — all Layer 1–4 images (~40–60 GB) + the in-box model (Ministral ~3 GB; a Magistral 24B would add ~15 GB) + image-churn headroom	**FIXED.** It does **not** grow with your dataset. Size it once for images + models.
Data storage	Object storage + PVCs	The **Iceberg lakehouse on object storage** (in-cluster MinIO for the demo → **STACKIT Object Storage / S3** for real scale → **TBs**), plus PVCs for **OpenSearch** (indices + embeddings), **Postgres** (metadata), **ClickHouse** (traces), **MLflow** (artifacts)	**Independently**, with the dataset. "More data" lands here.

The one gotcha worth internalizing: **don't confuse node RAM (128 GB) with the node disk** (the small volume that fills). The original 80 GB disk filled with images during deploy → **disk-pressure** → the node was **cordoned** → pods could no longer schedule. The fix is a **200 GB** node disk (`node_volume_size_gb`, now the Terraform default), sized for images + model weights with churn headroom — **not** for data.

And the corollary that keeps cost honest: **real data never touches the node disk.** It lives on separate, independently-scalable storage. When the dataset grows into the terabytes you grow the **object storage and the PVCs** — the node volume stays exactly where it is. More data ≠ a bigger node disk.

5.3 Durable lakehouse

On a long-lived deployment the lakehouse must survive pod restarts and node-rolls. Two backends hold the data and both are now persistent: **MinIO** keeps the Iceberg data files on a **PVC**, and **Polaris** keeps its catalog metadata in a **relational-JDBC metastore** (the bundled, PVC-backed Postgres) instead of in-memory — so the `lakehouse` warehouse registration is not lost on restart (otherwise Trino and the `query` tool report *"Unable to find warehouse lakehouse"* even with MinIO persisted). A **`polaris-catalog-init` Job** registers the `lakehouse` catalog on deploy, so queries resolve it with no manual step. The catalog is pinned to **Polaris 1.1.0-incubating** — the version the live Iceberg **write** path (the Data tab's physical Bronze/Silver/Gold builds) was verified against. (On a throwaway `kind` box the default stays ephemeral by design.)

5.4 Durable state — every store mirrors, one core

Persistence extends beyond the Iceberg lakehouse. **Every user-facing in-process store** in the OS UI — approvals, audit, artifacts, apps, agent systems (incl. `AGENT.md` / `MEMORY.md`), datasets, knowledge,

files, dashboards, big bets, users, domains, marketplace grants, pillars, preferences, role config — mirrors to **OpenSearch** through **one shared core,** `lib/os-mirror.ts`: fire-and-forget write-through on every change plus hydration on boot, so **artifacts survive redeploys and node-rolls** with no re-seeding.

The single shared core is deliberate: the earlier copy-pasted per-store pattern had a bootstrap bug (a missing index read as "mirror down forever"), which is exactly how artifacts were once lost on a deploy. The core fixes the semantics once for everyone — a missing index is **created**, an unreachable OpenSearch never breaks a request (the store simply stays in-memory), and an unhealthy mirror lazily re-probes and self-heals.

This requires the **OpenSearch PVC** (`openSearch.persistence.enabled: true`; migration for a live cluster: `deploy/opensearch-pvc-migration.sh`) — the default on STACKIT, disabled locally by default to save RAM. Without it the mirror itself dies with the pod and stores rebuild from seed data on restart. One honest residual gap: the in-process store stays authoritative, so writes made in the moments before a pod roll can be lost if their mirror write hadn't landed — see `docs/backups.md` for what protects the mirror itself.

5.5 Backups & restore

Durable is not the same as backed up — a PVC still dies with its disk. The STACKIT deploy adds a three-tier backup system (full detail: `docs/backups.md`; drills: `docs/runbooks/restore-drill.md`):

- **Nightly Postgres dumps** — a chart CronJob (`backup.pgDump`, on by default in `values.stackit-selfhosted.yaml`) dumps *every* database (Langfuse, LiteLLM, Dagster, Superset, OpenMetadata, Polaris, MLflow, Featureform, warehouse) to the lakehouse bucket, 14-day retention. This — not a file copy of the running database — is the consistent Postgres restore path.
- **Nightly Velero volume backups** — Velero + kopia (`deploy/velero/`) copies every stateful volume (MinIO lakehouse, OpenSearch mirrors, Forgejo repos, Harbor registry, …) **off-cluster** to a dedicated STACKIT Object Storage bucket, 30-day retention. Re-downloadable model weights are excluded by design.
- **The pre-upgrade gate** — every `helm upgrade` or stateful roll starts with `deploy/pre-upgrade-backup.sh`: a fresh dump plus an ad-hoc Velero backup, awaited before any change touches the platform.

Restore is practiced, not assumed: the restore-drill runbook restores each tier into a scratch namespace and verifies it, and lists honestly what is *not* protected (in-process-only writes before their next mirror, terraform state and operator-side secrets, scratch namespaces).

5.6 Security model

The platform ships **secure by default**. Four guarantees hold throughout:

- **Default-deny egress.** NetworkPolicies deny outbound traffic except DNS, intra-namespace traffic, and the API server; only the **egress proxy** may reach the internet, and it is allowlist-only and logs everything. (On `kind` the kindnet CNI doesn't enforce NetworkPolicies, so the app-layer chain — OPA → proxy → `web_fetch` — provides the guarantee; on STACKIT, **Cilium** enforces them and adds FQDN-aware allowlists and DLP.)

- **OPA tool authorization, least privilege.** A principal may invoke a tool only if granted; unknown principals and ungranted tools are denied. Internet access is an explicit grant — `web_fetch` is ungranted by default. Agents use a **scoped** virtual key with a spend cap, never the master key.
- **The web is data, not instructions.** The only path out is the governed `web_fetch` tool: OPA-authorized, routed through the egress proxy, and returned **as sanitized data** — never auto-written into the knowledge base. Retrieval results and tool output are treated the same way (the prompt-injection posture).
- **No real secret in git.** `.gitignore` blocks secret patterns; the chart ships only secure defaults. The local dev passwords in this guide exist only under `profile: local`. On STACKIT every secret lives in **Secrets Manager / KMS** and is synced by the **External Secrets Operator**; the chart references secrets by name only. Outbound-email secrets (the Microsoft Graph token / the SMTP password) are held the same way and are never logged or returned.

Every agent action — model calls, tool calls, retrievals — is **traced in Langfuse** with token cost and latency; outbound requests are logged at the egress proxy; telemetry/phone-home is disabled for a sovereign, offline posture.

Hardened in this release. Four fixes tightened the OS UI's own surface:

- **Every data-proxy API route requires a session.** Routes like `/api/query` used to be reachable un-authenticated; they now return **401 for anonymous callers** and scope results to the caller's domains via document-level security — one user can never read another domain's rows.
- **Policy checks fail closed.** If OPA is unreachable or errors, the governed-tool gate **denies** (with an explicit `opa-unreachable` marker) rather than waving the call through; fail-open exists only as an explicit opt-in (`OPA_FAIL_OPEN=true`) for the offline-mock teaching flow.
- **In-process stores are true singletons.** Every registry is pinned to `globalThis`, so Next.js route bundles can no longer each instantiate their own copy of a store (the bug that made `AGENT.md` / `MEMORY.md` intermittently 404).
- **Login/logout cache-busting.** Identity changes invalidate cached pages, so a signed-out browser never serves a stale, previously-authenticated view.

5.7 Models & the LLM gateway

Every model call goes through **LiteLLM** — the one gateway that enforces the model allowlist, per-key spend caps, tracing and graceful back-pressure. The self-hosted default routes are **free**; a live STACKIT deployment fronts two pay-per-token open-weight **Qwen** tiers on **STACKIT AI Model Serving**, matching the assistants' two phases:

- **Reasoning / planning** — `Qwen/Qwen3-VL-235B-A22B-Instruct-FP8` (LiteLLM `sovereign-reasoning`), the PLAN phase of every assistant.
- **Execution / default** — `Qwen/Qwen3.6-27B` (`sovereign-default`), tool-calling and general chat.

Qwen "thinking" is disabled (`chat_template_kwargs.enable_thinking=false`) so replies come back direct. The local **Ministral** model stays as an **offline fallback** — it currently wedges on this CPU node's llama.cpp warmup, so STACKIT Qwen is the live default. All STACKIT usage draws on one shared **€250/week** budget (LiteLLM `budget_duration: 7d`): once it is exhausted the gateway returns a **graceful budget error (HTTP 429)** rather than failing hard, and the pool auto-resets weekly.

WireGuard (optional, off by default). A first-class, durable in-cluster tunnel component (a UDP LoadBalancer) lets LiteLLM reach an **external** Ollama model — e.g. a Mac-Mini `qwen3:14b` on the `local-qwen` route — over a private link. Being part of the chart, it survives redeploys and node-rolls; enable it only when you want to bridge to an outside model.

5.8 Use the OS from Claude or ChatGPT (MCP)

The platform exposes itself as **governed MCP servers** — **live end-to-end** at `https://agentic.datamasterclass.com/api/mcp` — importable into Claude, ChatGPT, or any Streamable-HTTP MCP client. Open any tab's **"Connect your AI Tool via MCP"** button for a one-click import link with pre-filled instructions.

Connecting (OAuth flow). The server uses managed OAuth with the **client-id-metadata-document** pattern:

1. Your MCP client fetches the server's metadata document at the root (auto-handled by conforming clients such as Claude and ChatGPT).
2. The client is redirected (**303 consent redirect**) to the OS consent screen; you approve once per client.
3. The client receives a **180-day access token** — held in your client's credential store, never in the OS.
4. From that point every tool call is authenticated as you; the role floor is re-checked from the live session on every call, and OPA authorizes the tool. The token scope matches your OS role exactly — no broader.

One **overarching** server at the host above (Streamable-HTTP SSE, per-user token, role-scoped) surfaces the OS's cross-tab tools; alongside it, **per-tab** servers at `/api/mcp/<tab>` (`software`, `data`, `science`, `knowledge`, `agents`, `files`, `metrics`, `dashboards`, `bigbets`) each ship a token-minimal `CONTEXT.md`, so a client gets just that tab's tools and just enough context.

MCP is a full build surface, not a read-only window. Around fifty-five governed tools ship across the cross-tab server and the per-tab lenses — reads, writes, and **read-back parity** (everything a client can build, it can `list_*` / `get_*` back and verify):

- **Data** — the **physical pipeline**: `create_dataset` • `ingest_dataset` (upload → a real Bronze Iceberg table) • `transform_silver` • `build_gold_join` • `profile_dataset` • `add_dataset_version` • `document_dataset`, plus `query_data` (governed SQL)
- **The sharing ladder as tools** — `request_promotion` (the owner files; datasets & files) and `approve_promotion` (Builder+ in the domain applies; for datasets the approval **runs the physical publish**)
- **Knowledge** — `author_knowledge` • `publish_knowledge` • `index_knowledge` • `search_knowledge`
- **Files** — `upload_file` • `search_files` • `list_files` / `get_file`
- **Metrics** — `define_metric` • `query_metric` (resolve a governed metric to numbers, under RLS)
- **Dashboards** — `create_dashboard` • **Big Bets** — `create_big_bet` • `update_big_bet` • `attach_component`
- **Agents** — `create_agent_system` • `commit_agent_files` • `build_agent_system` • `run_agent_system`
- **Science** — `list_models` • `get_model` • `science_predict`

- **Connections** — `list` / `get` / `create` / `test_connection` + templates
- **Software** — the full app lifecycle (create • commit • preview • request deploy — the deploy *decision* stays Builder-gated)

Two **discovery tools** ship on every endpoint — `whoami` (who am I, which domains, what my role allows) and `list_capabilities` — and every failure returns a **typed, model-readable error** `{ code, reason, hint }` so a client can self-correct instead of guessing.

The same front door serves the platform's **own agents**: an Agent-tab system's tool calls dispatch through this identical governed toolset under its owner's identity — there is no separate, privileged internal registry.

> **Not yet exposed via MCP** (designed, not built): Strategy, the Marketplace, the Governance approvals queue, Monitoring, Platform-Admin, and the Science lifecycle beyond reads + predict (promote/certify/drift/retrain). The list above is the real, shipped surface.

Every tool runs **as the signed-in user**: the per-user token carries your identity, the **role floor is re-checked from the session on every call** (never trusted from the request body), OPA authorizes it, and document-level security scopes what you see. Promotion-class actions — `promote_*`, `publish_knowledge`, and the Software deploy decision — stay **Builder/Admin-gated**, exactly as in the UI. Every MCP tool delegates to the **same governed library function the UI calls** — OPA policy, role gates and Langfuse audit apply unchanged, and there is no privileged path.

5.9 The live teaching cohort

The live STACKIT deployment doubles as the classroom for the **Agentic Leader Program (Q3 2026)**, and its setup is a worked example of the whole operating model:

- **Two domains.** `agentic-leader-q3-2026` hosts the cohort — the instructor as **Builder** plus **36 participants as Creators** (each signs in with their **email as username**) — and a separate `test` domain keeps instructor dry-runs out of the cohort's space. The user roster and all credentials live only in the gitignored private values overlay, never in the repository.
- **The Campaign-Optimization exercise.** The Northpeak Commerce case study is seeded **domain-Shared** into `agentic-leader-q3-2026` through the platform's **own governed endpoints** (`deploy/apply-campaign-exercise.sh`) — campaign datasets, three knowledge documents, sample campaign files, a ready-made **Campaign Evaluation Agent**, and a **Campaign App**. Because the materials are Shared, every participant can *use and run* them — but as Creators they cannot edit the shared artifacts or promote their own work without a Builder, so the exercise teaches the promotion ladder by living inside it.

5.10 Memory and the build-and-toggle defaults

The full L1+L2+L3 set is sized for a large STACKIT node. To fit a 14 GB local VM, a few heavy components ship **off** by default locally (Docling, OpenSearch Dashboards, OpenMetadata, Spark, the Layer-4 Science stack, plus Terminal and Workbench). Turn any of them on from **Platform → Components** (a runtime

on/off that scales 0↔1) or permanently by setting `<component>.enabled: true` and re-running `helm upgrade`. *Off* means installed but scaled to zero; *disabled* means not deployed at all.

5.11 Components at a glance

Grouped by layer — see `docs/components/<id>.md` for the full per-component guide.

LAYER	COMPONENTS
L1 — Agent core	LiteLLM (model + MCP gateway, per-key budget cap) • model-server (self-hosted Ministral 3 / Magistral 24B; two-tier STACKIT Qwen reasoning/execution on the live deploy) • mock-model (offline embeddings + fallback) • OpenSearch (retrieval) • Langfuse (tracing) • query-tool (Trino MCP) • system agents (Domain RAG • ML pipeline • Hermes runtime)
L2 — Foundations	OPA • Docling • Haystack • Dagster • dbt • Cube • OpenMetadata
Infra	Postgres (CloudNativePG) • ClickHouse • Valkey • MinIO (object storage, PVC-backed) • Polaris (Iceberg catalog, durable relational-JDBC metastore)
L3 — Self-service	central Trino • Superset • Forgejo (sovereign git) • Argo CD • CI runner/build • OpenSearch Dashboards • Workbench (workload only — retired from the nav) • Terminal
L4 — Science	JupyterHub • MLflow • Featureform • KServe (all opt-in)
Security & platform	egress-proxy • web_fetch • WireGuard tunnel (optional) • OS UI (embedded Components console • same-origin tool proxy + Level-1 SSO • remote & per-tab MCP servers)

5.12 Full demo-login table (profile `local`)

CONSOLE	PORT-FORWARD (`kubectl -n agentic-os …`)	URL	LOGIN
OS UI	`port-forward svc/os-ui 8080:3000`	`http://localhost:8080`	–
Langfuse	`port-forward svc/agentic-os-langfuse-web 3000:3000`	`http://localhost:3000`	`admin@datamasterclass.com` / `langfuse-local-dev-admin`
LiteLLM	`port-forward svc/agentic-os-litellm 4000:4000`	`http://localhost:4000/ui`	`admin` / `litellm-admin-local-dev`
Superset	`port-forward svc/agentic-os-superset 8088:8088`	`http://localhost:8088`	`admin` / `superset-admin-local-dev`
Forgejo	`port-forward svc/forgejo-http 3001:3000`	`http://localhost:3001`	`gitea_admin` / `forgejo-admin-local-dev`
Argo CD	`port-forward svc/argocd-server 8082:80`	`http://localhost:8082`	`admin` / secret `argocd-initial-admin-secret`
MinIO	`port-forward svc/minio 9001:9001`	`http://localhost:9001`	`agentic-os-local` / `agentic-os-local-secret`
Cube	`port-forward svc/cube 4001:4000`	`http://localhost:4001`	– (playground)
Dagster	`port-forward svc/agentic-os-dagster-webserver 3070:80`	`http://localhost:3070`	–
OpenMetadata*	`port-forward svc/openmetadata 8585:8585`	`http://localhost:8585`	`admin@open-metadata.org` / `admin`
Polaris	`port-forward svc/polaris 8181:8181`	`http://localhost:8181`	root / `polaris-local-dev-secret` (OAuth2)
OpenSearch (API)	`port-forward svc/opensearch 9200:9200`	`http://localhost:9200`	–

*Off by default locally — enable first (Platform → Components toggle, or `enabled: true` + `helm upgrade`).

5.13 Troubleshooting

- **`ImagePullBackOff` on `demo-app` right after install** is expected — it clears once the first CI run builds and bumps the image tag.
- **Out of memory / pods pending.** The slice is RAM-bound — keep the heavy components off locally or give the VM more RAM.
- **Where do I log in first?** Locally the OS UI needs no login; Langfuse (`admin@datamasterclass.com` / `langfuse-local-dev-admin`) is the default admin-style back-end console.
- **Agent answers look canned** → the self-hosted model (`model-server`) is disabled, so the offline mock model is answering. Enable `modelServer` or point LiteLLM at any model — no agent change.
- **`web_fetch` returns 403 / 502** → 403 means OPA hasn't granted the principal `web_fetch`; 502 means the domain isn't on the egress allowlist. Both are by design.
- **No verification/invite email is sent** → no mailer is configured. The bootstrap admin **auto-verifies**, so you can start without one; to enable email, set the Microsoft Graph app-registration (`Mail.Send`) **or** the `SMTP_*` variables (sender `support@datamasterclass.com`). Note: on the live STACKIT tenant outbound delivery is currently **not operational** (provider port-25 block; relay + DNS pending) — see *Getting started*.
- **Do I have to use STACKIT?** No — any Kubernetes works; the chart is portable. STACKIT is the sovereign EU default. You can build and validate everything on local `kind` with no cloud key.

5.14 Version & changelog

- **Chart 0.2.12 • app `0.2.0-alpha.12` • os-ui `0.1.44`.** This build: generated `2026-07-07` from commit `a2ad0fe`.
- **This build (os-ui 0.1.44): Tier-1 platform hardening + MCP live.** MCP is **live end-to-end** at `https://agentic.datamasterclass.com/api/mcp` with managed OAuth (client-id-metadata-document flow, 303 consent redirect, Streamable-HTTP SSE, 180-day access token) and governed per-user tool execution. **Real file storage and download** (MinIO PVC-backed). **User-invite flow**: the platform now generates a one-time temporary password the Administrator shares out-of-band; invitee sets their own on first login. **Cockpit/Home** domain-pulse and spend feeds are live (no longer mock-stubbed on a deployed cluster). **Monitoring** alerts, cost and health feeds are live (no more canned fixtures). **Marketplace**: connection templates create real governed Connections; deploy-instance produces a real running artifact. **Big Bets** link real cross-tab artifacts and real metric values; Strategy↔Big Bets linkage is live. **Agents**: promote UX on the system card; scheduling provisions a real Kubernetes CronJob. **Dashboards**: real Superset import, and scheduled report + alert delivery are live. **Data**: row-preview (governed inline sample) and honest not-materialized state labels. **Admin**: single configurable assistant LLM (STACKIT managed model endpoint/key) that powers all built-in artifact-building assistants across every tab; **Google Drive and OneDrive** OAuth app registration (client ID + secret) for the Files connector, live. Honest state: the governance spine (OPA, approvals, RLS, promote ladders, roles, audit, MCP, auth, Knowledge, Data pipeline) is fully live;

external tool execution beyond Drive/OneDrive, the in-cluster Software live-app runner, and Science/Layer-4 are still being wired or deferred.

- **os-ui 0.1.32: the role model grew to four ranks** — `creator < builder < domain_admin < admin`. The new **Domain admin** carries everything a Builder can **plus** administering the users of their own domain(s) only (invite, edit, deactivate, roles up to Builder — never another Domain admin or an Administrator; only the platform Administrator appoints Domain admins). Builders are approvers, **not** people-admins. Governance stays Builder-rank-and-up; the rest of the Platform group stays Administrator-only; nobody is auto-promoted on upgrade. **Durability shipped**: every user-facing in-process store now mirrors to OpenSearch through the one shared `lib/os-mirror.ts` core (write-through + boot hydration, self-healing index bootstrap) — artifacts survive redeploys; requires the OpenSearch PVC. **The Data golden path went physical end to end** (Data M1): upload → a real Bronze Iceberg table in a per-user schema → Explore → Silver → Gold join → **publish-on-approval** (the approval runs the physical publish) → Cube → **Talk to your data v2** (governed NL→SQL: canView-scoped context, one validated read-only SELECT, executed under the caller's row filters), on Polaris 1.1.0-incubating. **MCP Waves A + B**: the physical pipeline tools, the `request_promotion` / `approve_promotion` split, `query_metric`, `run_agent_system`, Science reads (`list_models` / `get_model`), Big Bet updates, Connections tools and read-back parity (`list_*` / `get_*` for every buildable artifact) — and internal Agent-tab systems now dispatch through the **same governed toolset** under their owner's identity. **Nav consolidation**: Tutorials in the main tab group; Governance atop the Platform group (Builder+); Workbench retired from the nav (the workload remains chart-optional). **Console UX**: Terminal auto-connects and re-attaches to a live session; Dagster's public ingress gained operator basic-auth. **Backups**: the Tier 0–2 system (nightly pg-dump CronJob • nightly Velero off-cluster volume backups • the pre-upgrade backup gate) with honest gap documentation in `docs/backups.md` and drills in `docs/runbooks/`.
- **Earlier (os-ui 0.1.16).** The role model was consolidated to three roles — `creator < builder < admin` (*participant* and *agentic-leader* removed; agentic-leaders migrated to Creator) — and **Users admin** was overhauled: invite by email (email = username), multi-select domain membership, role descriptions, edit, and archive → restore → permanently-delete behind confirmations. **MCP became a full build surface**: per-tab governed **write tools** on every endpoint (data create/version/document/promote • knowledge author/publish/index • files upload/promote • define_metric • create_dashboard • create_big_bet • agent create/commit/build • the Software lifecycle), plus `whoami` + `list_capabilities` discovery and typed `{code, reason, hint}` errors — every call as the signed-in user (per-user token, OPA/DLS, session-checked role floor), promotion/publish/deploy still Builder/Admin-gated. The **Software Delivery Team** shipped — a six-agent LangGraph system (orchestrator • planner • builder • tester • deployer • communication; builder on `sovereign-default`, the rest on `sovereign-reasoning`) run by a per-user graph executor, with `decide_deploy` withheld so go-live stays a human Builder decision. The **Agents tab was rebuilt**: React-Flow drag-and-drop graph builder, master-detail rail, one Auto/Reasoning/Execution model toggle with real `sovereign-*` names and internal/external badges, reliable AGENT.md/MEMORY.md (globalThis store fix); the poet demo is gone and the backend service agents (Domain RAG • ML pipeline • Hermes) moved to the Platform tab as system agents, with guided Hermes runtime cards. **Security lockdown 2**: session-required + DLS-scoped data-proxy routes (`/api/query` et al.), fail-closed OPA in `governed.ts`, all in-process stores globalThis-pinned, login/logout cache-busting. Every tab header gained the **top-left ActionBar** (Tutorial + "Connect your AI Tool via MCP"). The live tenant now runs the **Agentic Leader Q3-2026 cohort** (36 Creators + instructor-Builder in `agentic-`

`leader-q3-2026`, plus a `test` domain) with the **Campaign-Optimization (Northpeak) exercise seeded domain-Shared** through the governed endpoints.

- **os-ui 0.1.4.** Every tab assistant is now **agentic** — PLAN (reasoning tier) → act/codegen (execution tier, tool-calling) → verify → **gated deploy** — not just chat; the Software build assistant genuinely scaffolds → commits → previews → requests deploy. The OS is importable into **Claude/ChatGPT as governed MCP**: one authenticated remote server at `/api/mcp` plus per-tab servers at `/api/mcp/<tab>`, each re-using the UI's governed functions. Back-end tools now embed **same-origin** through the OS UI (`/tools/<tool>`) with **Level-1 header SSO** that auto-provisions per-role accounts. The **lakehouse is durable** — MinIO on a PVC, Polaris on a relational-JDBC (Postgres) metastore, and a `polaris-catalog-init` Job that registers the `lakehouse` catalog on deploy. On STACKIT, LiteLLM serves a **two-tier Qwen** stack (`sovereign-reasoning` = Qwen3-VL-235B, `sovereign-default` = Qwen3.6-27B; thinking disabled) under a shared **€250/week** budget cap with graceful 429, local Ministral kept as offline fallback; an optional durable **WireGuard** tunnel can bridge LiteLLM to an external Ollama model. New in this guide pass: **Platform → MCPs & APIs** (four-group governed tool-registry with per-entry Claude/ChatGPT import links); **Northpeak Commerce live metrics** — `iceberg.sales.gold_northpeak_commerce` materialized via Trino CTAS backs the `northpeakcommerce` Cube model (four measures × three dimensions, sliceable without SQL); **durable state** for the Data/Metrics registries and Admin→Domains (OpenSearch mirroring + boot hydration; requires OpenSearch PVC).
- **0.2.x** — the **OS UI v1.0**: every sidebar tab a real surface, brand-themed, light/dark, with the operational console embedded at Platform → Components; Layers 1–3 in place under the secure-by-default baseline; Science (L4) opt-in. This release's **UI rework**: **Home** is now the golden-path launcher only and the live cockpit modules moved to a new **Cockpit** tab (with a top-items-per-artifact board); **Strategy** is exactly three sections (Big Bets • Self Service • Foundations); **Agents** and **Software** are each one page (Software on sovereign in-cluster Forgejo — no accounts/tokens); **Monitoring** is artifact-observability only while infra health moved to **Components**; **Connections** is external-systems only; **Platform Admin** is labelled **Admin**; terminology is **Personal** + **domain** throughout; the left-nav was reordered (Home, Cockpit, Strategy, Big Bets, Agents, Software, Science, Knowledge, Files, Data, Metrics, Dashboards, Connections, Marketplace, Monitoring, Governance, Settings); and first-run gained an **auto-verifying bootstrap admin** with optional email via **Microsoft Graph** `sendMail` or **SMTP**.
- **Next** — real external tool execution beyond Drive/OneDrive; the in-cluster live-app runner for Software; broader tutorial anchors; full per-domain spaces; Science/Layer-4 wiring. Bump this section additively as the OS evolves and re-run `scripts/build-docs.sh`.

Sovereign Agentic OS — built from permissively-licensed open source for EU data residency. This guide is generated from the repository; to update it, edit `docs/Sovereign-Agentic-OS-Guide.md` and run `scripts/build-docs.sh`.